1.Write a python program to perform tokenization by word and sentence using NLTK

Description:

This Python program demonstrates how to perform sentence tokenization and word tokenization using the NLTK (Natural Language Toolkit) library. It begins by installing and importing the required NLTK package and downloading the necessary tokenizer resources. A sample text is then provided as input, which is processed using sent_tokenize() to split the paragraph into individual sentences and word_tokenize() to divide the text into individual words and punctuation marks. Finally, the program displays the tokenized sentences and words, showing how tokenization helps in preprocessing text for various Natural Language Processing (NLP) applications.

Code:

```
# Install NLTK (Run this only once in Google Colab)
!pip install nltk

# Import NLTK
import nltk

# Download required tokenizer
nltk.download('punkt')
nltk.download('punkt_tab')   # Required for newer NLTK versions

# Import tokenizers
from nltk.tokenize import word_tokenize, sent_tokenize

# Input text
text = """
Natural Language Processing (NLP) is a branch of Artificial Intelligence.
It helps computers understand human language.
Python provides the NLTK library for NLP tasks.
Tokenization is the first step in text preprocessing.
"""

# --------------------------------
# Sentence Tokenization
# --------------------------------
sentences = sent_tokenize(text)

print("Sentence Tokenization:")
for i, sentence in enumerate(sentences, start=1):
    print(f"{i}. {sentence}")

# --------------------------------
# Word Tokenization
# --------------------------------
```

```
words = word_tokenize(text)

print("\nWord Tokenization:")
print(words)
```

2.Write a python program to eliminate stop words using NLTK.

Description:

This Python program removes stop words from a sentence using the NLTK (Natural Language Toolkit) library. It imports the required modules, downloads the necessary datasets (punkt and stopwords), and tokenizes the input sentence into individual words. The program then compares each word with the predefined list of English stop words and removes words such as "is," "the," "a," "an," and "to", which do not contribute much meaning to text analysis. Finally, it displays the original sentence and the filtered list of words. This technique is commonly used in Natural Language Processing (NLP), text mining, sentiment analysis, search engines, and machine learning applications.

Code:

```
# Import libraries
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download required NLTK data (Run only once)
nltk.download('punkt')
nltk.download('stopwords')

# Input sentence
text = "This is a simple example to remove stop words using NLTK."

# Tokenize the sentence
words = word_tokenize(text)

# Load English stop words
stop_words = set(stopwords.words('english'))

# Remove stop words
filtered_words = [word for word in words if word.lower() not in stop_words]
```

```
# Display results
print("Original Sentence:")
print(text)

print("\nAfter Removing Stop Words:")
print(filtered_words)
```

3.Write a python program to perform parts of speech (POS) tagging using NLTK.

Description:

This Python program performs Part-of-Speech (POS) tagging using the NLTK library. It tokenizes the input sentence into words and assigns a grammatical tag to each word, such as noun, verb, adjective, or adverb using the pos_tag() function. Finally, it displays each word along with its corresponding POS tag. POS tagging is widely used in Natural Language Processing (NLP), text analysis, machine translation, and information extraction.

Code:

```
# Import libraries
import nltk

# Download required NLTK data (Run only once)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

from nltk.tokenize import word_tokenize
```

```
print("\nLemmatized Words:")
print(lemmatized_words)
```

5.write a python program to perform chucking using NLTK.

Description:

This Python program performs chunking using the NLTK library. It tokenizes the input sentence, assigns Part-of-Speech (POS) tags to each word, and uses a chunk grammar to group related words into noun phrases (NP). Finally, it displays the POS tags and the chunked sentence. Chunking is commonly used in Natural Language Processing (NLP), information extraction, and text analysis.

Code

```
# Import libraries
import nltk
from nltk import word_tokenize, pos_tag, RegexpParser

# Download required NLTK data (Run only once)
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')

# Input sentence
text = "The quick brown fox jumps over the lazy dog."

# Tokenize the sentence
words = word_tokenize(text)

# Perform POS tagging
tagged_words = pos_tag(words)

# Define chunk grammar
grammar = "NP: {<DT>?<JJ>*<NN>}"

# Create chunk parser
chunk_parser = RegexpParser(grammar)

# Perform chunking
chunk_tree = chunk_parser.parse(tagged_words)
```

```
# Display the results
print("POS Tags:")
print(tagged_words)

print("\nChunk Tree:")
print(chunk_tree)

# Draw the chunk tree (optional) - commented out as it requires a display
environment
# chunk_tree.draw()
```

6. Write a python program to perform spam filter using NLTK.

Description:

This Python program performs a simple spam filter using the NLTK library. It tokenizes the input message into individual words and compares them with a predefined list of spam keywords such as "free," "lottery," "prize," and "claim." If any spam keyword is found, the message is classified as Spam; otherwise, it is classified as Not Spam. This technique is commonly used in email filtering, SMS filtering, and Natural Language Processing (NLP) applications.

```
if score['compound'] >= 0.05:
   print("Sentiment: Positive")
elif score['compound'] <= -0.05:
   print("Sentiment: Negative")
else:
   print("Sentiment: Neutral")
```

8.Write a python program to detect fake news using NLTK.

Description:

This Python program performs fake news detection using the NLTK library. It tokenizes the input news article into individual words and compares them with a predefined list of fake news keywords such as "breaking," "shocking," "miracle," and "guaranteed." If any of these keywords are found, the news is classified as Fake News; otherwise, it is classified as Real News. This simple approach is useful for learning basic Natural Language Processing (NLP) concepts.

Code:

```
# Import libraries
import nltk

# Download required NLTK data (Run only once)
nltk.download('punkt')
nltk.download('punkt_tab')

from nltk.tokenize import word_tokenize

# Input news
news = "The government announced a new education policy to improve digital
learning across schools."



# Convert to lowercase and tokenize
words = word_tokenize(news.lower())

# List of fake news keywords
fake_keywords = ["breaking", "shocking", "miracle", "guaranteed", "secret", "fake",
"rumor", "click", "hidden", "truth"]

# Detect fake news
```

```
if any(word in fake_keywords for word in words):
    print("News:", news)
    print("Result: Fake News")
else:
    print("News:", news)
    print("Result: Real News")
```

9. Write a python program to Named Entity Recognition(NER) using NLTK .

Description:

This Python program performs Named Entity Recognition (NER) using the NLTK library. It tokenizes the input sentence, assigns Part-of-Speech (POS) tags, and uses the ne_chunk() function to identify named entities such as persons, organizations, and locations. Finally, it displays the named entity tree. NER is widely used in Natural Language Processing (NLP), information extraction, search engines, chatbots, and text analysis.

Code:

```
bigrams = list(ngrams(words, 2))

# Generate Trigrams
trigrams = list(ngrams(words, 3))

# Display the results
print("Unigrams:")
print(unigrams)

print("\nBigrams:")
print(bigrams)

print("\nTrigrams:")
print(trigrams)
```